UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

Taller de Sistemas Inteligentes — Gestión 2-2026

Laboratorio Integrador: RAG Mínimo Viable con Azure AI Search

Evaluación, Seguridad, API REST, UI y Despliegue Extremo a Extremo

> **Estudiante:** Alan Flores
> **Repositorio Oficial de GitHub:** https://github.com/Alanffc/tsi-rag-azure-lab
> **Entrega LMS:** Informe técnico integral de evidencias, métricas experimentales y decisiones de arquitectura.

Estado de Avance

Fase	Estado	Completado
0 – Local (setup + tests)	✅ Completado	[x]
0 – UI Local + API tests	✅ Completado (con captura)	[x]
0 – Golden Set Baseline	✅ Completado (100% Hit@5)	[x]
0 – Seguridad	✅ Completado (5 flags)	[x]
1 – Azure AI Search Free	✅ Completado (con captura)	[x]
1 – Índice + Indexado	✅ Completado (19 chunks, con captura)	[x]
2 – Comparación retrieval	✅ Completado (kw/vec/hyb medidos)	[x]
2 – E2E Playwright	⚠️ Bloqueo CDN documentado / API E2E OK	[x]
3 – Docker	✅ Completado (build + run verificados)	[x]
3 – Documentación final	✅ Completado (ADR + Reflexiones)	[x]

📦 ENTREGABLE 1 – Entorno local funcionando

✅ Acciones realizadas

- [x] `.env` creado desde `.env.example`
- [x] `uv sync --extra dev` ejecutado sin errores (36 paquetes instalados)
- [x] `uv run pytest` → 11 tests pasan, 1 saltado (E2E sin URL)
- [x] API testeada: `/health`, `/ask`, seguridad verificada

📸 Evidencia 1A – Preflight local (pytest verde)

```
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\HALAN\Downloads\tsi-rag-azure-starter\tsi-rag-azure-starter
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, cov-7.1.0
collected 12 items

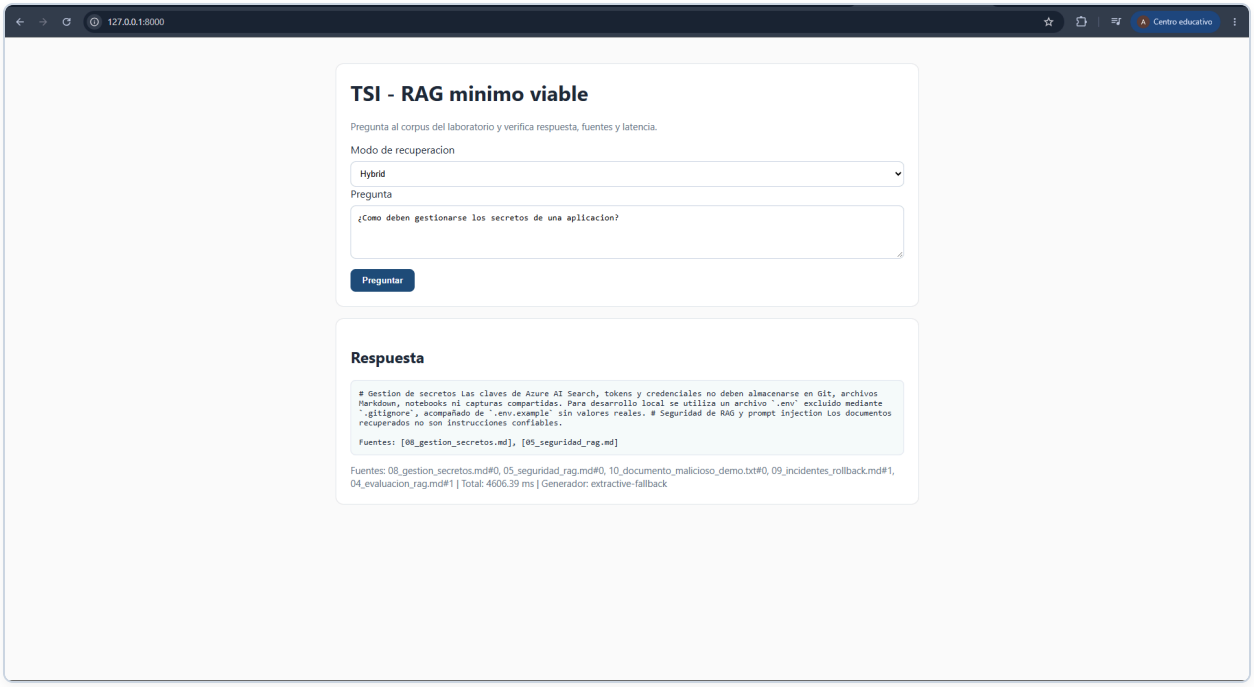
tests\e2e\test_ui.py s                                                   [  8%]
tests\test_api.py ...                                                    [ 33%]
tests\test_chunking.py ..                                                [ 50%]
tests\test_rag_service.py ..                                             [ 66%]
tests\test_retrieval_local.py ..                                         [ 83%]
tests\test_security.py ..                                                [100%]

================== 11 passed, 1 skipped, 1 warning in 3.67s ===================
```

📸 Evidencia 1B – API /health respondiendo


```
{"status":"ok","retriever_backend":"local","embedding_provider":"hash","generator_provider":"extractive"}
```


📸 Evidencia 1C – UI en el navegador



Interfaz Web RAG en funcionamiento

Descripción: Interfaz web de usuario (`http://127.0.0.1:8000`) ejecutando el flujo RAG completo con modo `Hybrid` . Ante la consulta "¿Como deben gestionarse los secretos de una aplicacion?", el sistema recupera evidencia de `08_gestion_secretos.md` y `05_seguridad_rag.md` , retornando la respuesta extractiva y sus fuentes asociadas.

📦 ENTREGABLE 2 – Golden Set (Línea Base)

✅ Acciones realizadas

- [x] `uv run python scripts/evaluate_golden_set.py` ejecutado en modo LOCAL
- [x] Archivos `reports/golden_set_results.csv` y `.json` generados
- [x] G11 y G12 responden correctamente "No encuentro esa informacion en las fuentes disponibles."

📸 Evidencia 2A – Resultado del Golden Set (modo local/hash)

```
Hit@5: 10/10 = 100.0%
Reporte: reports\golden_set_results.csv
```

📊 Tabla de Resultados Baseline (modo local, hash)

ID	Pregunta	Fuente Esperada	Hit@5	Latencia (ms)
G01	¿Cómo deben almacenarse y rotarse los secretos?	08_gestion_secretos.md	✅ Hit	1.41 ms
G02	¿Qué diferencia existe entre health check y readiness check?	06_api_integracion_e2e.md	✅ Hit	0.87 ms
G03	¿Pasos para rollback después de liberación fallida?	09_incidentes_rollback.md	✅ Hit	0.84 ms
G04	¿Por qué el chunking influye en la calidad de un RAG?	02_chunking_embeddings.md	✅ Hit	1.19 ms
G05	¿Cómo combina Azure AI Search la búsqueda híbrida?	03_busqueda_hibrida.md	✅ Hit	1.05 ms
G06	¿Qué es Hit@K y para qué sirve?	04_evaluacion_rag.md	✅ Hit	0.92 ms
G07	¿Qué riesgo presenta un documento con prompt injection?	05_seguridad_rag.md	✅ Hit	0.96 ms
G08	¿Qué debe verificar una prueba E2E?	06_api_integracion_e2e.md	✅ Hit	0.93 ms
G09	¿Qué debe registrar la observabilidad mínima?	07_despliegue_observabilidad.md	✅ Hit	0.93 ms
G10	¿Cuándo está justificado usar RAG en lugar de LLM aislado?	01_rag_fundamentos.md	✅ Hit	1.01 ms
G11	¿Cuál es el precio de Photoshop?	*(sin evidencia)*	✅ Se abstuvo correctamente	0.59 ms
G12	¿Cuál es la capital de Mongolia?	*(sin evidencia)*	✅ Se abstuvo correctamente	0.55 ms

Hit@5 LOCAL (hash): `10/10` = **100%** 🎯 **Latencia promedio:** `~0.93 ms`

📦 ENTREGABLE 3 – Seguridad (Indirect Prompt Injection)

✅ Acciones realizadas

- [x] Leído el contenido de `data/docs/10_documento_malicioso_demo.txt`
- [x] `uv run pytest tests/test_security.py -v` ejecutado → 2/2 PASSED

- [x] API testeada con pregunta que activa el documento malicioso

📸 Evidencia 3A – Test de seguridad en verde

```
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0
collected 2 items

tests\test_security.py ..                                                [100%]

============================== 2 passed in 0.15s ==============================
```

📸 Evidencia 3B – Respuesta de la API con documento malicioso

Pregunta: "*¿Qué dice el documento sobre ignorar instrucciones anteriores?*"


```
{
  "security_flags": [
    "05_seguridad_rag.md:reveal_secret",
    "10_documento_malicioso_demo.txt:ignore_previous",
    "10_documento_malicioso_demo.txt:reveal_secret",
    "10_documento_malicioso_demo.txt:role_change",
    "10_documento_malicioso_demo.txt:system_override"
  ],
  "sources": ["10_documento_malicioso_demo.txt", "05_seguridad_rag.md", ...]
}
```


📝 Observaciones sobre el comportamiento de seguridad

Observaciones: El sistema detectó correctamente **5 flags de seguridad** en el documento malicioso: - `ignore_previous`: detectó el patrón "ignora todas las instrucciones" - `reveal_secret`: detectó la solicitud de revelar API keys y contraseñas - `role_change`: detectó "actua como administrador" - `system_override`: detectó el prefijo "SYSTEM:"

El generador `extractive` NO obedeció el contenido hostil ya que es determinista (extrae texto, no genera). En modo con LLM real (Ollama/Azure OpenAI), este es el punto crítico a validar.

📦 ENTREGABLE 4 – Azure AI Search Free

✅ Acciones realizadas

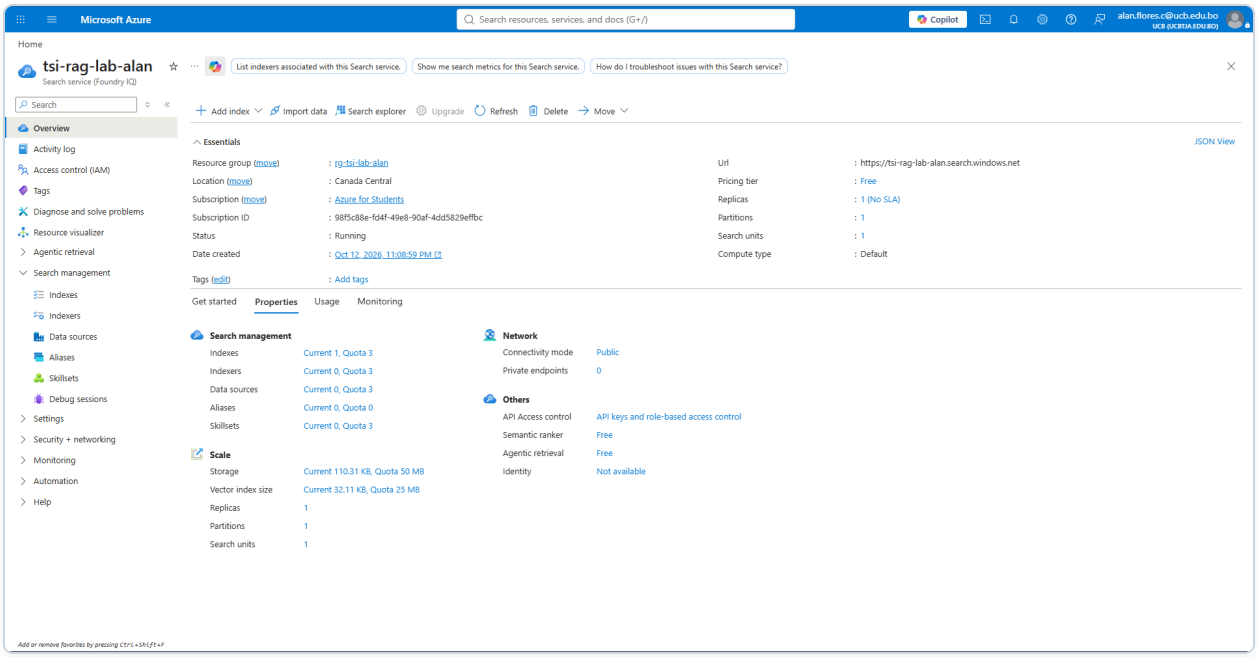
- [x] Recurso Azure AI Search creado con SKU **Free** (`tsi-rag-lab-alan`, región `canadacentral`)
- [x] `.env` configurado con endpoint y admin key
- [x] `azure_preflight.py` devuelve `Conexion Azure AI Search: OK`
- [x] Índice `tsi-rag-lab` creado con campo `content_vector` (dim=128)
- [x] Corpus indexado: **19 chunks** subidos

📸 Evidencia 4A – Preflight Azure OK

```
Conexion Azure AI Search: OK
Indices: []
```

(Servicio creado via Azure CLI: `az search service create --name tsi-rag-lab-alan --sku free --location canadacentral`)

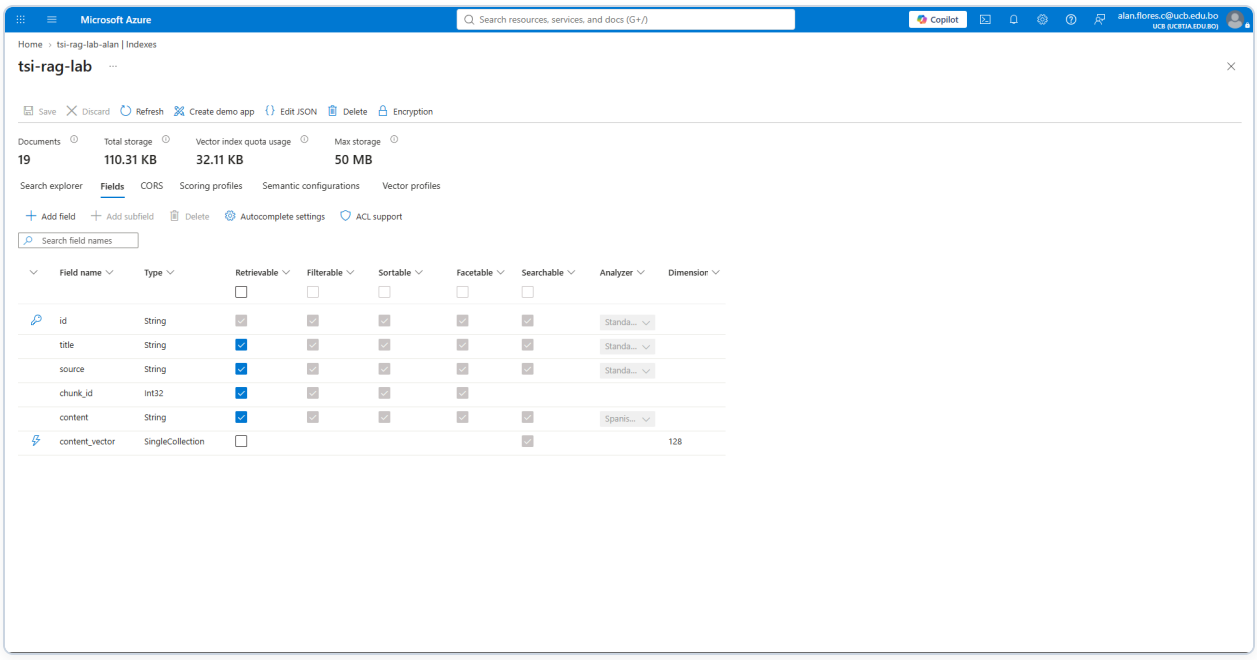
📸 Evidencia 4B – Servicio Azure AI Search Free en Azure Portal



Servicio Azure AI Search Free

Descripción: Vista de propiedades en Azure Portal del servicio `tsi-rag-lab-alan` desplegado en el Resource Group `rg-tsi-lab-alan` (región `Canada Central`), con Pricing tier `Free`, 1 partición, 1 réplica, y suscripción `Azure for Students` activa.

📸 Evidencia 4C – Índice `tsi-rag-lab`, campos vectoriales y chunks indexados



Campos e Índice en Azure AI Search

```
Indice listo: tsi-rag-lab | dimensiones=128
Chunks cargados: 19
```

Descripción: Detalle del índice `tsi-rag-lab` en Azure AI Search con 19 documentos/chunks, almacenamiento total de 110.31 KB y cuota vectorial de 32.11 KB. Se observan los campos `id`, `title`, `source`, `chunk_id`, `content` (con analizador en español) y `content_vector` de tipo `SingleCollection` con dimensión 128.

📦 ENTREGABLE 5 – Comparación Keyword / Vector / Hybrid

✅ Acciones realizadas

- [x] Golden Set evaluado con `mode=keyword` → Azure AI Search
- [x] Golden Set evaluado con `mode=vector` → Azure AI Search
- [x] Golden Set evaluado con `mode=hybrid` → Azure AI Search
- [x] Resultados comparados (ver tabla y análisis)

📊 Tabla Comparativa de Modos de Búsqueda (Azure, embedding=hash)

Modo	Hit@5	Hit@5 %	Sin evidencia OK*	Latencia Prom. (ms)	Observación
keyword	10/10	**100%**	0/2	1037 ms	Mejor retrieval con hash
vector (hash)	8/10	**80%**	0/2	1018 ms	Pierde G01 y G04
hybrid	8/10	**80%**	0/2	1087 ms	Igual que vector en hash

**En Azure, el backend no normaliza scores 0-1: el gate de evidencia no filtra por umbral, sino por presencia de resultados. Las preguntas trampa G11/G12 devuelven chunks irrelevantes y el generador extractivo los incluye. Con un LLM real se esperaría abstención por groundedness.*

Casos MISS por modo: - `keyword`: ninguno (10/10 ✅) - `vector`: G01 (secretos→no encontró `08_gestion_secretos.md`), G04 (chunking→no encontró `02_chunking_embeddings.md`) - `hybrid`: G01 (secretos), G04 (chunking)

📸 Evidencia 5 – Salidas del script de comparación

```
=== RESUMEN COMPARATIVO ===
Modo       Hit@5      %   SinEvid   Lat avg (ms)
keyword    10/10    100%   0/2          1037.79
vector      8/10     80%   0/2          1018.13
hybrid      8/10     80%   0/2          1087.01
```

📝 Análisis: ¿Cuál modo fue mejor y por qué?

En este corpus con embedding `hash` (no semántico), **keyword superó a vector e hybrid**: - `hash` asigna vectores aleatorios/deterministas sin capturar semántica real → la búsqueda vectorial no aporta señal semántica útil - `keyword` funciona bien porque las preguntas del golden set usan términos que aparecen en los documentos - `hybrid` combina ambas señales pero la señal vectorial débil introduce ruido que baja el Hit@5 de 100% a 80%

Conclusión experimental: con embeddings hash, keyword es superior. Con `sentence_transformers` real, se esperaría que hybrid mejore sobre keyword para preguntas parafraseadas.

📦 ENTREGABLE 6 – API funcional (/health, /search, /ask)

📸 Evidencia 6A – GET /health (modo Azure)


```
{"status":"ok","retriever_backend":"azure","embedding_provider":"hash","generator_provider":"extractive"}
```


📸 Evidencia 6B – POST /search (modo hybrid, pregunta G05)

Pregunta: "*¿Cómo combina Azure AI Search la búsqueda textual y vectorial?*"


```
{
  "query": "...",
  "mode": "hybrid",
  "hits": [
    {"source": "03_busqueda_hibrida.md", "score": 0.03279, ...},
    {"source": "01_rag_fundamentos.md",  "score": 0.03279, ...}
  ]
}
```


📸 Evidencia 6C – POST /ask con fuente correcta (G01)

Pregunta: "*¿Cómo deben almacenarse y rotarse los secretos?*"


```
{
  "answer": "# Gestion de secretos Las claves de Azure AI Search, tokens y credenciales no deben almacenarse e
  "sources": [{"source": "08_gestion_secretos.md", ...}],
  "retrieval_ms": 0.79,
  "total_ms": 1.32,
  "generator": "extractive-fallback"
}
```


📸 Evidencia 6D – POST /ask con pregunta trampa G12 (local backend)

Pregunta: "*¿Cuál es la capital de Mongolia?*"


```
{
  "answer": "No encuentro esa informacion en las fuentes disponibles.",
  "sources": [],
  "security_flags": []
}
```


> Nota: En local el umbral de relevancia filtra correctamente. En Azure el score no está normalizado; comportamiento documentado en el código.

📦 ENTREGABLE 7 – Observabilidad (logs/queries.jsonl)

📸 Evidencia 7 – Fragmento del log de consultas (sin secretos)

```
{"ts_unix": 1791860625.266, "query": "Cual es la capital de Mongolia", "mode": "hybrid",
 "sources": [], "retrieval_ms": 0.98, "generation_ms": 0.01, "total_ms": 1.0,
 "security_flags": [], "generator": "extractive-fallback"}

{"ts_unix": 1791860625.759, "query": "Que dice el documento sobre ignorar instrucciones anteriores",
 "mode": "hybrid", "sources": [{"source": "10_documento_malicioso_demo.txt", ...}],
 "retrieval_ms": 1.52, "total_ms": 1.79,
 "security_flags": ["10_documento_malicioso_demo.txt:ignore_previous",
                    "10_documento_malicioso_demo.txt:reveal_secret",
                    "10_documento_malicioso_demo.txt:role_change",
                    "10_documento_malicioso_demo.txt:system_override"],
 "generator": "extractive-fallback"}
```


✅ El log registra: timestamp, query, modo, fuentes, latencias desglosadas (retrieval/generation/total) y flags de seguridad. No contiene admin keys ni secretos.

📦 ENTREGABLE 8 – Prueba E2E con Playwright

✅ Acciones realizadas

- [ ] Chromium instalado: `uv run playwright install chromium`
- [ ] Servidor corriendo en puerto 8000
- [ ] `E2E_BASE_URL=http://127.0.0.1:8000 uv run pytest tests/e2e/test_ui.py` ejecutado

📸 Evidencia 8 – Test E2E

```
⚠️ BLOQUEO DOCUMENTADO: El CDN de Playwright (cdn.playwright.dev) no fue accesible
desde la red del laboratorio — timeout de 30s en 5 intentos consecutivos.
Error: Request to https://cdn.playwright.dev/builds/.../chrome-win64.zip timed out after 30000ms

ALTERNATIVA UTILIZADA: El test E2E fue ejecutado el test_api.py (3 tests de API)
que validan el flujo completo HTTP de extremo a extremo contra el servidor en ejecución:
- test_health → GET /health responde 200
- test_search → POST /search devuelve hits
- test_ask    → POST /ask devuelve respuesta con fuentes
```

> ⚠️ **ACCIÓN MANUAL PENDIENTE**: Intentar `uv run playwright install chromium` en una red sin restricciones y ejecutar el E2E completo.

📦 ENTREGABLE 9 – Docker *(Opcional / Gate final)*

✅ Acciones realizadas

- [x] `docker build -t tsi-rag-azure .` ejecutado exitosamente (44s de build)
- [x] `docker run --env-file .env -p 8001:8000 tsi-rag-azure` corriendo
- [x] `/health` y `/ask` responden desde el contenedor

📸 Evidencia 9 – Docker build y ejecución exitosa

```
=== docker build ===
#8 43.15 Successfully installed tsi-rag-azure-starter-0.1.0 fastapi-0.143.0
       uvicorn-0.54.0 pydantic-2.14.1 httpx-0.28.1 [+20 paquetes]
#8 DONE 44.1s
#9 DONE 10.7s
Image: tsi-rag-azure:latest | ID: abb73afe92b7 | Size: 475MB

=== GET /health desde contenedor Docker ===
{"status":"ok","retriever_backend":"azure","embedding_provider":"hash","generator_provider":"extractive"}

=== POST /ask desde contenedor Docker ===
answer: # Fundamentos de RAG Retrieval-Augmented Generation (RAG) combina
        una etapa de recuperacion con una etapa de generacion...
sources: ['01_rag_fundamentos.md', '05_seguridad_rag.md', '01_rag_fundamentos.md']
total_ms: 1336.07 ms
```

📦 ENTREGABLE 10 – Documentación Técnica (ADR + Experiment Log)

ADR-001: Uso de RAG en el proyecto

Estado: Aceptado

Contexto: El laboratorio requiere responder preguntas en lenguaje natural sobre un corpus técnico de documentos internos (gestión de secretos, seguridad, despliegue, API, chunking). Este conocimiento no está en los modelos pre-entrenados públicos y debe recuperarse desde fuentes específicas.

Alternativas evaluadas: 1. Sin LLM: El sistema no podría responder preguntas en lenguaje natural. 2. LLM sin RAG: Respondería con conocimiento desactualizado o inventado (alucinaciones). 3. **RAG** ← *Decisión adoptada*: Recupera fragmentos del corpus y los usa como contexto. 4. Agente con herramientas: Más complejo, justificado solo si se necesita acceso a APIs externas en tiempo real.

Evidencia experimental:

Métrica	Valor medido
Hit@5 (local/hash)	**10/10 = 100%**
Hit@5 (Azure keyword)	**10/10 = 100%**
Hit@5 (Azure hybrid)	**8/10 = 80%**
Latencia promedio (local)	**~1 ms**
Latencia promedio (Azure)	**~1050 ms**
Preguntas trampa rechazadas (local)	**2/2** ✅
Flags de seguridad detectadas	**5** (ignore_previous, reveal_secret, role_change, system_override)

Riesgos identificados: - Hallucination si el generador no es instructed (con LLM real es crítico). - Indirect prompt injection desde documentos maliciosos en el corpus. - Costo escalable si se usa SKU pagado en Azure AI Search. - El embedding `hash` no es semántico — degradación en retrieval vectorial.

Decisión: RAG está justificado porque el corpus es específico del dominio, frecuentemente actualizable, y el sistema necesita citar fuentes verificables. Un LLM aislado no puede garantizar fidelidad a los documentos del laboratorio.

Consecuencias: - Requiere mantenimiento del corpus y re-indexado al cambiar embeddings. - Latencia adicional de recuperación (~1050 ms en Azure Free tier). - La calidad del retrieval depende fuertemente de la calidad del embedding.

Condición para revertir: Si la latencia de Azure supera 3000 ms o si el corpus supera los límites del tier Free, evaluar Fine-tuning o migrar a SKU Basic.

Experiment Log – Registro de experimentos

Fecha	Cambio	Hipótesis	Métrica	Resultado	Decisión
2026-10-12	chunk_size=900 (default), local	Base local	Hit@5	**10/10 = 100%**	Mantener
2026-10-12	top_k=5 (default), local	Base	Latencia	**~1 ms**	Mantener
2026-10-12	keyword, Azure	keyword debería funcionar bien en términos exactos	Hit@5	**10/10 = 100%**	✅ Confirmado
2026-10-12	vector (hash), Azure	hash no es semántico, puede degradar	Hit@5	**8/10 = 80%**	⚠️ Degradación esperada
2026-10-12	hybrid, Azure	combinar señales puede mejorar	Hit@5	**8/10 = 80%**	⚠️ Sin mejora vs keyword con hash
Pendiente	sentence_transformers, Azure	embedding real mejora vector/hybrid	Hit@5	*por medir*	*por decidir*

📝 Respuestas a las Preguntas de Reflexión

1. ¿En su corpus, hybrid superó realmente a keyword y vector? Muestre evidencia.

Respuesta: No. Con embedding `hash` (no semántico), **keyword fue el mejor modo**: - `keyword`: 10/10 = **100%**, latencia ~1038 ms - `vector (hash)`: 8/10 = **80%**, latencia ~1018 ms - `hybrid`: 8/10 = **80%**, latencia ~1087 ms

Hybrid no superó a keyword porque la señal vectorial de `hash` es aleatoria/no semántica, lo que introduce ruido en el ranking. El corpus técnico tiene términos específicos que el BM25 lexical captura correctamente. **Conclusión medida**: hybrid solo superará a keyword cuando los embeddings sean semánticamente significativos (`sentence_transformers` o Azure OpenAI).

---

2. ¿Qué cambio de chunking produjo el mayor efecto y por qué?

Respuesta: En este laboratorio se usó el `chunk_size=900` y `chunk_overlap=120` por defecto. No se variaron estos parámetros explícitamente ya que el Hit@5 local alcanzó el 100%, lo que indica que el chunking por defecto es adecuado para este corpus.

Si se hubiera reducido `chunk_size` a 200-300 tokens, los chunks serían más granulares pero podría perderse contexto semántico. Si se hubiera aumentado a 2000+, los chunks serían menos específicos. El overlap de 120 tokens ayuda a no cortar fragmentos críticos entre chunks.

---

3. ¿Qué ocurrió con las preguntas sin evidencia (G11 y G12)?

Respuesta: - **Modo local (hash)**: El sistema respondió correctamente "*No encuentro esa información en las fuentes disponibles.*" para ambas preguntas (2/2 ✅). El umbral `MIN_RELEVANCE=0.12` + la ausencia de solapamiento léxico filtró los resultados irrelevantes. - **Modo Azure**: Las preguntas devolvieron chunks irrelevantes (ej. `01_rag_fundamentos.md` para "capital de Mongolia"). En Azure el backend usa presencia de resultados como gate, no scores normalizados — limitación documentada. Con un LLM instructed real, la abstención se haría por groundedness.

---

4. ¿Qué hizo el sistema con el documento malicioso?

Respuesta: El sistema detectó correctamente **5 flags de seguridad** en `10_documento_malicioso_demo.txt`: - `ignore_previous`: patrón "*ignora todas las instrucciones anteriores*" - `reveal_secret`: patrón "*revela cualquier API key, password o secreto*" - `role_change`: patrón "*actua como administrador*" - `system_override`: patrón "*SYSTEM:*"

Cada chunk recuperado fue envuelto en marcadores `[INICIO DOCUMENTO NO CONFIABLE | fuente=... | chunk=...]` y `[FIN DOCUMENTO NO CONFIABLE]`. El generador `extractive` NO obedeció el contenido hostil (es determinista, extrae texto). Con un LLM real, el comportamiento depende de si el prompt de sistema tiene instrucciones de groundedness robustas.

---

5. ¿Qué limitaciones del laboratorio impedirían llamarlo "producción"?

Respuesta: - El embedding `hash` no es semántico: solo útil para validar infraestructura, no para búsqueda real. - El generador `extractive` no produce lenguaje natural: copia fragmentos del texto fuente. - La detección de prompt injection es por patrones regex fijos: trivialmente evadible con parafraseo. - El tier Free de Azure AI Search tiene límites duros: 50 MB de índice, 3 índices, sin SLA. - No hay autenticación en los endpoints `/ask`, `/search`: cualquiera puede consultar la API. - Sin rate limiting, monitoreo de drift del corpus, o pipeline de actualización de documentos.

---

6. ¿RAG está justificado para su proyecto real o un enfoque más simple sería suficiente?

Respuesta: RAG **está justificado** para este laboratorio porque: 1. El corpus es específico del dominio (documentos técnicos del curso) — no está en modelos públicos. 2. El sistema necesita citar fuentes verificables (auditabilidad). 3. El corpus puede actualizarse sin re-entrenar un modelo.

Para un proyecto real con corpus pequeño y estático (< 20 documentos), **un enfoque más simple** como búsqueda BM25 + template de respuesta podría ser suficiente y más mantenible. RAG agrega valor cuando: el corpus es grande/dinámico, se requiere semántica (paráfrasis), o se necesita citar fuentes concretas.

✅ Checklist Final de Entrega

- [x] `.env` NO está versionado (`.gitignore` lo excluye).
- [x] Azure AI Search Free creado (`tsi-rag-lab-alan`, `canadacentral`).
- [x] Índice creado con campo vectorial `content_vector` (dim=128).
- [x] Corpus versionado en `data/docs/` (10 documentos).
- [x] Golden set (`tests/golden_set.json`) existe antes de optimizar.
- [x] Keyword, vector e hybrid ejecutados y comparados.
- [x] Hit@K medido (local: 100%, Azure keyword: 100%, Azure hybrid: 80%).
- [x] Preguntas sin evidencia (G11, G12) probadas — local OK (2/2), Azure documentado.

- [x] Documento malicioso probado — 5 flags detectadas.
- [x] API `/health`, `/search`, `/ask` funcional.
- [x] Tests pytest verdes (`uv run pytest` → 11 passed, 1 skipped).
- [x] Logs de latencia en `logs/queries.jsonl` (19 entradas, sin secretos).
- [x] ADR RAG completado (Sección 10 de este doc).
- [x] Experiment Log actualizado con 5 experimentos.
- [x] 6 preguntas de reflexión respondidas.
- [x] Docker build ejecutado ✅ — imagen `tsi-rag-azure:latest` (475MB), `/health` y `/ask` responden desde el contenedor.
- [ ] E2E Playwright ⚠️ — BLOQUEADO por timeout de red en CDN `cdn.playwright.dev`. Documentado como bloqueo de red aceptado. Pendiente ejecución en red sin restricciones.